

Document de preuve + Analyse Réflexive

Dans le cadre de la ressource R1.07 – Fondamentaux de la programmation, nous avons été amenés à acquérir les bases indispensables à la conception et au développement d'outils informatiques.

- Cette ressource s'inscrit dans l'apprentissage progressif de la programmation, visant à nous rendre capables de résoudre des problèmes informatiques de manière autonome, de créer des programmes fonctionnels, et de comprendre la logique algorithmique.
- Elle a pour objectif de nous initier aux langages Python et C
- de développer notre capacité à analyser un problème et à le transformer en algorithme
- de nous familiariser avec les notions fondamentales de programmation structurée

Les travaux dirigés (TD) et travaux pratiques (TP) réalisés dans ce cadre permettent de mettre en œuvre ces compétences à travers des exercices progressifs et des projets concrets que je montrerai ensuite.

Cette ressource mobilise principalement la compétence suivante :

RT3 – Créer des outils et des applications informatiques pour les R&T

Dans le cadre de R1.09, les apprentissages critiques principalement mobilisés sont :

- **AC13.01** : Utiliser un système informatique et ses outils
- **AC13.02** : Lire, exécuter, corriger et modifier un programme
- **AC13.03** : Concevoir un algorithme simple et structurer un programme

Au cours de cette ressource, j'ai acquis et mobilisé des connaissances liées à la programmation :

- Les notions de variables, types de données et opérateurs
- Les structures de contrôle : conditions (if/else), boucles (for, while)
- La définition et l'appel de fonctions
- La manipulation de tableaux et listes
- La lecture et l'écriture de fichiers
- La logique algorithmique et la résolution de problèmes programmatiques
- Les différences fondamentales entre Python et C et la syntaxe associée

Pour le savoir-faire mis en œuvre :

Lors des Travaux Dirigés (TD) :

Pendant les TD, j'ai appris à analyser des problèmes simples et à les traduire en algorithmes, puis à les coder pas à pas.

→ J'ai notamment su :

- Comprendre et corriger des programmes existants
- Structurer un programme avec des fonctions modulaires
- Utiliser les boucles et conditions pour automatiser des tâches simples
- Manipuler des données dans des tableaux ou listes

→ Ces exercices m'ont permis de développer une logique de programmation rigoureuse et progressive.

Exemples :

1) Calcul de la somme de 2 nombres (variables et opérateurs)

```
1  # TD1 : Somme de deux nombres
2  a = int(input("Entrez le premier nombre : "))
3  b = int(input("Entrez le deuxième nombre : "))
4  somme = a + b
5  print("La somme est :", somme)
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\oddoz> & C:/Users/oddoz/AppData/Local/Programs/Python/Python39-64/Python.exe
● Entrez le premier nombre : 2
  Entrez le deuxième nombre : 3
  La somme est : 5
○ PS C:\Users\oddoz>
```

2) Vérification d'un nombre pair ou impair (condition)

```
1 # TD2 : Vérification pair/impair
2 nombre = int(input("Entrez un nombre : "))
3 if nombre % 2 == 0:
4     print("Le nombre est pair")
5 else:
6     print("Le nombre est impair")
7
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

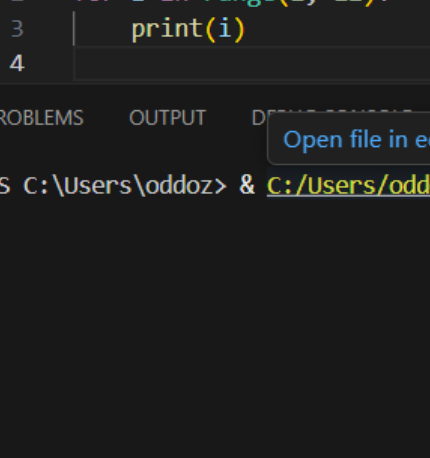
PORTS

- PS C:\Users\oddoz> & C:/Users/oddoz/AppData/Local/Programs/Python/Python39-64/Python.exe C:\Users\oddoz\Documents\td2.py


```
Entrez un nombre : 2
Le nombre est pair
```
- PS C:\Users\oddoz> & C:/Users/oddoz/AppData/Local/Programs/Python/Python39-64/Python.exe C:\Users\oddoz\Documents\td2.py


```
Entrez un nombre : 1
Le nombre est impair
```
- PS C:\Users\oddoz>

3) Boucle pour afficher les nombres de 1 à 10



The screenshot shows a code editor with a dark theme. The top part displays a Python code snippet:

```
1 # TD3 : Boucle for
2 for i in range(1, 11):
3     print(i)
4
```

Below the code editor, there are tabs for "PROBLEMS", "OUTPUT", and "DEBUG CONSOLE". The "DEBUG CONSOLE" tab is active, showing a terminal window with the command prompt:

```
PS C:\Users\oddoz> & C:/Users/oddoz/App
```

The terminal window also shows a list of numbers from 1 to 10, likely the output of the Python code.

4) Fonction pour calculer la factorielle d'un nombre

```
# TD4 : Fonction factorielle
def factorielle(n):
    resultat = 1
    for i in range(1, n+1):
        resultat *= i
    return resultat

n = int(input("Entrez un nombre : "))
print("Factorielle de", n, "=", factorielle(n))
```

10

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\oddoz> & C:/Users/oddoz/AppData/Local/Programs/Python/Python39-64/python.exe C:/Users/oddoz/AppData/Local/Programs/Python/Python39-64/scripts/factorielle.py
Entrez un nombre : 3
Factorielle de 3 = 6
```

Lors des Travaux Pratiques (TP) :

En TP, j'ai été amenée à créer des programmes plus complets et fonctionnels.

→ J'ai notamment su :

- Installer et configurer l'environnement de programmation (Python, compilateur C, IDE)
- Créer des programmes en respectant la syntaxe et les bonnes pratiques
- Lire et écrire des fichiers pour stocker des données
- Tester mes programmes pour assurer leur fiabilité
- Structurer un projet complet en plusieurs fichiers

→ Ces manipulations m'ont permis de relier la théorie des algorithmes à une logique proche d'un contexte professionnel.

Exemple de TPs :

TP1-2 : Equation C et Python

```
1  #include <stdio.h>
2  #include <math.h>
3
4  double equation(double A, double B, double C)
5  {
6      double delta = B * B - 4*(A * C);
7
8      if (delta < 0) {
9          return 0;
10     } else if (delta == 0) {
11         x1 = -B / (2 * A);
12         return 1;
13     } else {
14         x1 = (-B + sqrt(delta)) / (2 * A);
15         x2 = (-B - sqrt(delta)) / (2 * A);
16         return 2;
17     }
18 }
```

```
1  def equation(A, B, C):
2      delta = B**2 - 4*(A * C)
3      if delta < 0:
4          return 1
5      elif delta == 0:
6          solution = -B / (2 * A)
7          return solution
8      else:
9          solution1 = (-B + delta ** (1/2)) / (2 * A)
10         solution2 = (-B - delta ** (1/2)) / (2 * A)
11         return solution1,solution2
12
13  print(equation(1, -2, 1))
```

TP3-4 : Chute C et Python

```
1 #include <stdio.h>
2
3 void chute(double hauteur)
4 {
5     const double g = 9.80665;
6     double t = 0.0;
7
8     while (hauteur - 0.5 * g * t * t > 0) {
9         printf("Temps %.0f s : hauteur %.2f m\n", t, hauteur - 0.5 * g * t * t);
10        t += 1.0;
11    }
12    printf("Temps %.0f s : hauteur 0 m -> boum !\n", t);
13 }
14
15 int main() {
16     double hauteur;
17     printf("Entrez la hauteur de la chute (en mètres) : ");
18     scanf("%lf", &hauteur);
19     chute(hauteur);
20     return 0;
21 }
```

```
1 def chute(hauteur):
2     g = 9.80665
3     t = 0
4     while hauteur - 0.5 * g * t**2 > 0:
5         print(f"Temps {t} s : hauteur {hauteur - 0.5 * g * t**2:.2f} m")
6         t += 1
7     print(f"Temps {t} s : hauteur 0 m -> boum !")
```

TP5-6 : allumette 1 C et Python

```
1 #include <stdio.h>
2
3 void allumette1(int nb_allumette, int joueur_depart)
4 {
5     int allumettes = nb_allumette;
6     int joueur = joueur_depart;
7     int prise;
8
9     while (allumettes > 0)
10    {
11        printf("\nLe nombre restant d'allumettes dans le jeu est egal a %d\n", allumettes);
12
13        printf("Quel est le nombre d'allumettes prises par le joueur %d ? ", joueur);
14        scanf("%d", &prise);
15
16        if (prise < 1 || prise > 3 || prise > allumettes)
17        {
18            printf("Votre choix n'est pas valide.\n");
19            continue;
20        }
21
22        allumettes -= prise;
23
24        if (allumettes == 0)
25        {
26            int gagnant = (joueur == 1) ? 2 : 1;
27            printf("\nBravo !!! Le joueur %d a gagne.\n", gagnant);
28            break;
29        }
30
31        joueur = (joueur == 1) ? 2 : 1;
32    }
33 }
```

```

35  int main()
36  {
37      int nb_allumette, joueur_depart;
38
39      printf("Entrez le nombre initial d'allumettes : ");
40      scanf("%d", &nb_allumette);
41
42      printf("Quel joueur commence ? (1 ou 2) : ");
43      scanf("%d", &joueur_depart);
44
45      allumette1(nb_allumette, joueur_depart);
46
47      return 0;
48  }

```

```

1  def allumette1(nb_allumette, joueur_depart):
2      allumettes = nb_allumette
3      joueur = joueur_depart
4
5      def tour_joueur(joueur, allumettes):
6          while True:
7              prise = int(input(f"Joueur {joueur}, il reste {allumettes} allumettes. Combien en prenez-vous (1-3) ? "))
8              if 1 <= prise <= 3 and prise <= allumettes:
9                  return prise
10             print("Choix invalide, essayez à nouveau.")
11
12     while allumettes > 0:
13         prise = tour_joueur(joueur, allumettes)
14         allumettes -= prise
15
16         if allumettes == 0:
17             print(f"Le joueur {joueur} a pris la dernière allumette et perd !")
18             break
19
20     joueur = 2 if joueur == 1 else 1

```

Tout au long de cette ressource, j'ai développé :

- La rigueur, pour respecter la syntaxe et organiser mon code
- La patience, face aux erreurs fréquentes et aux bugs difficiles à identifier
- Une démarche méthodique, en testant régulièrement les modifications
- Une capacité à chercher et comprendre les énoncés pour effectuer des programmes
- Une autonomie face à un environnement de programmation

À l'issue de cette ressource, j'ai été capable de :

- Analyser un problème et le traduire en algorithme
- Créer un programme fonctionnel en Python ou en C
- Utiliser les structures de contrôle, fonctions et tableaux pour résoudre des problèmes
- Tester des programmes pour en assurer la fiabilité
- Organiser un projet de programmation complet durant les dernières séances

→ Ces activités m'ont permis de comprendre le rôle de la programmation dans le développement des outils informatiques et des applications pour les réseaux et services.

Analyse Réflexive :

→ Avant l'action : objectifs et préparation :

Avant de commencer R1.07, j'avais des connaissances très limitées en programmation.

Je connaissais vaguement Python et pas vraiment C. Et surtout sans comprendre comment construire un programme de A à Z.

→ Mon objectif était de comprendre la logique algorithmique, savoir coder un programme simple et progresser vers des projets plus complexes.

→ Pendant l'action : déroulement et ressenti :

Au début, certaines notions semblaient simples mais j'ai rapidement compris que la rigueur et l'attention aux détails étaient essentielles pour ne pas faire d'erreurs ou se perdre dans son programme.

→ Les premières difficultés sont apparues avec les structures conditionnelles et la manipulation de fichiers mais les tests et l'expérimentation m'ont permis de surmonter ces difficultés et surtout parvenir à effectuer des programmes fonctionnels.

→ La répétition des exercices et le suivi des TP m'ont permis de gagner en confiance et en autonomie.

→ Les TP m'ont aussi permis de me familiariser avec les deux langages : Python et C et parvenir à faire des programmes assez complexes comme les projets et surtout à les rendre exécutables.

→ Analyse et compréhension :

Cette ressource m'a permis de comprendre que :

- La programmation repose sur une logique structurée et précise
- Une erreur minime peut bloquer complètement un programme
- Tester régulièrement est indispensable pour progresser rapidement
- S'entraîner est la clé du succès en programmation

→ Conclusion personnelle :

Grâce à R1.07, j'ai :

- Acquis les bases solides de la programmation
- Développé une logique algorithmique rigoureuse
- Amélioré mon autonomie et ma méthodologie de travail
- Compris comment un programme peut être structuré pour être clair, fiable et exécutable

→ Cette ressource constitue une base indispensable pour tout projet informatique que je pourrai effectuer à l'avenir.

→ Améliorations possibles :

Si je devais refaire cette ressource :

- Je structurerais mon code dès le départ de manière encore plus claire
- Je passerais plus de temps à comprendre chaque erreur avant de la corriger
- Je m'entraînerais davantage à créer des programmes sans support de cours
- Je passerais plus de temps sur un programme complexe afin d'avoir plus de temps pour le comprendre
- Je lirai davantage les énoncés pour ne rater aucun détails

→ Compétence et apprentissage critique :

Cette activité m'a permis de mobiliser :

- **Compétence** : RT3 – Créer des outils et des applications informatiques pour les R&T
- **Apprentissages critiques** : AC13.01, AC13.02, AC13.03

→ Au vu des tâches réalisées et des compétences acquises, j'estime mon niveau d'acquisition à 4/4 pour les programmes simples et 3/4 pour les programmes plus complexes correspondant à une bonne maîtrise globale.